

Exp: 30 RTOS Task Scheduling

Program:

```
// Code your testbench here.

// Uncomment the next line for SystemC modules.

// #include <systemc>

#include <systemc.h>

SC_MODULE(rtos_scheduler) {

void task1() {

while(true) {

cout << "Task1 Running at "

<< sc_time_stamp() << endl;

wait(1, SC_SEC);

}

}

void task2() {

while(true) {

cout << "Task2 Running at "

<< sc_time_stamp() << endl;

wait(1, SC_SEC);

}

}

SC_CTOR(rtos_scheduler) {

SC_THREAD(task1);

SC_THREAD(task2);

}

};

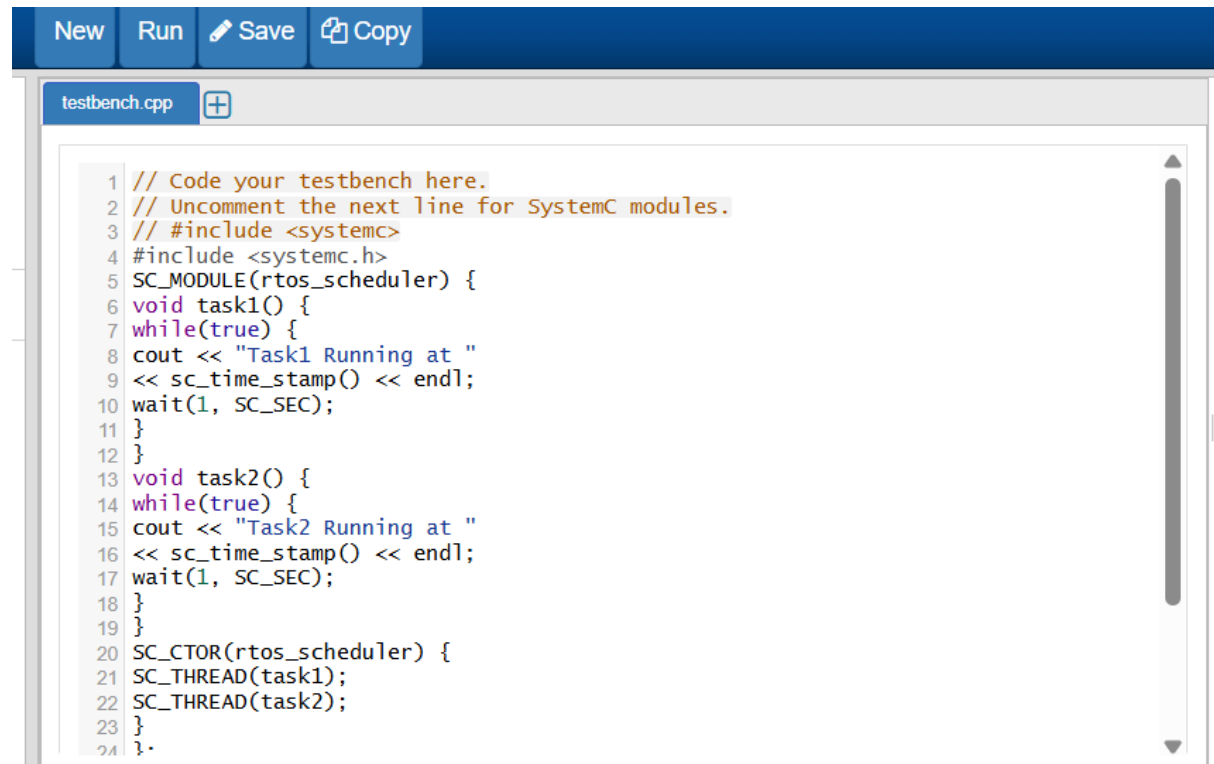
int sc_main(int argc, char* argv[]) {

rtos_scheduler obj("Scheduler");
```

```
sc_start(6, SC_SEC);
```

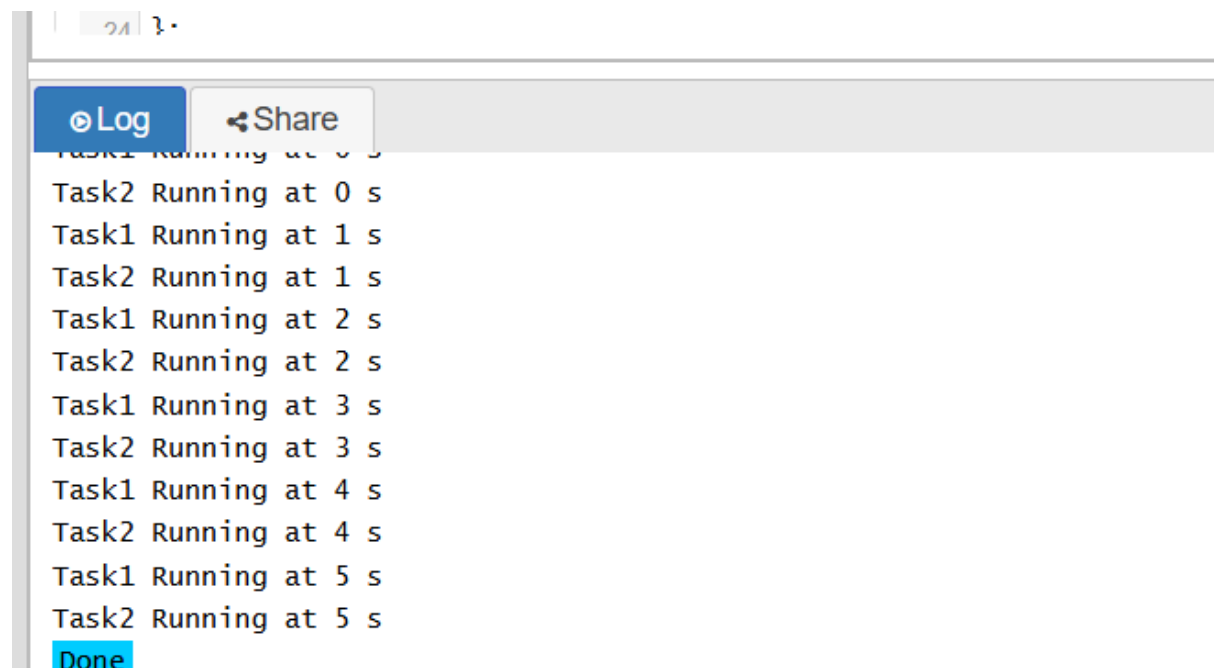
```
return 0;
```

```
}
```



```
1 // Code your testbench here.
2 // Uncomment the next line for SystemC modules.
3 // #include <systemc>
4 #include <systemc.h>
5 SC_MODULE(rtos_scheduler) {
6 void task1() {
7 while(true) {
8 cout << "Task1 Running at "
9 << sc_time_stamp() << endl;
10 wait(1, SC_SEC);
11 }
12 }
13 void task2() {
14 while(true) {
15 cout << "Task2 Running at "
16 << sc_time_stamp() << endl;
17 wait(1, SC_SEC);
18 }
19 }
20 SC_CTOR(rtos_scheduler) {
21 SC_THREAD(task1);
22 SC_THREAD(task2);
23 }
24 }
```

Output:



```
Task1 Running at 0 s
Task2 Running at 0 s
Task1 Running at 1 s
Task2 Running at 1 s
Task1 Running at 2 s
Task2 Running at 2 s
Task1 Running at 3 s
Task2 Running at 3 s
Task1 Running at 4 s
Task2 Running at 4 s
Task1 Running at 5 s
Task2 Running at 5 s
Done
```